

## Actividad: Desde las expresiones regulares hasta los DFA

Tema: 2.4 — Desde las expresiones regulares hasta los DFA Libro base: Construcción de Compiladores: Principios y Práctica — Kenneth C. Louden

Objetivo general:

Comprender el proceso completo mediante el cual una expresión regular puede transformarse en un autómata finito determinista (DFA), listo para implementarse en código dentro de un analizador léxico.

Contexto teórico

En la construcción de compiladores, el analizador léxico convierte la entrada (código fuente) en una secuencia de tokens.

Para lograrlo, las expresiones regulares que describen los patrones de los tokens se transforman en autómatas finitos deterministas (DFA). Este proceso se realiza en tres etapas fundamentales:

1. Expresión regular → NFA (método de Thompson)
2. NFA → DFA (construcción por subconjuntos)
3. Minimización del DFA → versión más eficiente para implementación en código

### Actividad práctica

Parte 1 — De expresión regular a NFA (Reglas de Thompson)

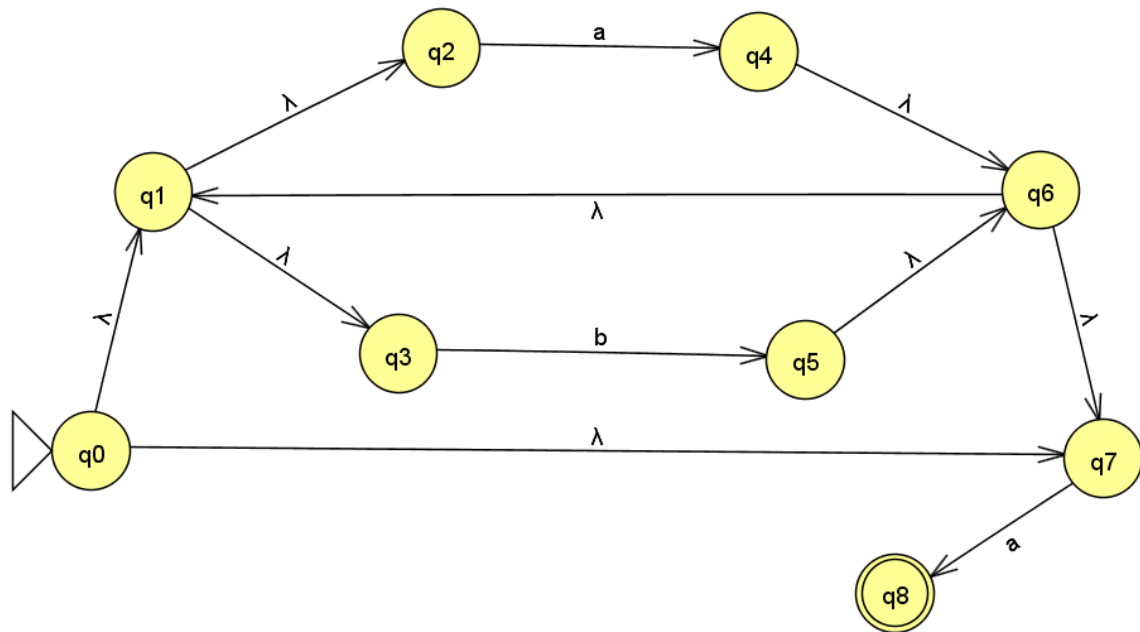
Instrucción:

Aplica las reglas de Thompson para construir el NFA que representa la siguiente expresión regular:

$R = (a|b)^*aR = (a|b)^*a$

1. Dibuja el NFA mostrando todas las transiciones (incluye las transiciones  $\epsilon$ ).

Los épsilon se representan en jflap como lambdas.



2. Describe cómo el NFA reconoce las cadenas:

- a
- ba
- bba

#### Cadena: "a"

1. Estado inicial: q0
2. Se usa una **transición  $\epsilon$**  para saltar al bloque final de la a (q7)
3. Se consume el símbolo a  $\rightarrow$  se llega a q8
4. q8 es estado de aceptación  $\rightarrow$  la cadena **es aceptada**.

#### Cadena: "ba"

1. Estado inicial: q0
2.  $\epsilon \rightarrow$  q1 (inicio del bloque (a|b)<sup>\*</sup>)
3. Elegimos rama de b: q1  $\xrightarrow{\epsilon}$  q4  $\xrightarrow{b}$  q5
4.  $\epsilon \rightarrow$  q6  $\rightarrow \epsilon \rightarrow$  q7 (salida del bloque repetible)

5.  $q7 \xrightarrow{a} q8$
6.  $q8$  es aceptación  $\rightarrow$  la cadena **es aceptada**.

**Cadena: "bba"**

1. Estado inicial:  $q0$
2.  $\epsilon \rightarrow q1$
3. Primera b:  $q1 \xrightarrow{\epsilon} q2 \rightarrow q3 \xrightarrow{b} q5 \rightarrow \epsilon \rightarrow q6 \rightarrow \epsilon \rightarrow q1$  (repetimos)
4. Segunda b:  $q1 \xrightarrow{\epsilon} q3 \rightarrow q3 \xrightarrow{b} q5 \rightarrow \epsilon \rightarrow q6 \rightarrow \epsilon \rightarrow q7$  (salimos del bloque repetible)
5.  $q7 \xrightarrow{a} q8$
6.  $q8$  es aceptación  $\rightarrow$  la cadena **es aceptada**.

3. ¿Qué función cumplen las transiciones  $\epsilon$  en este proceso?

Representa una transición vacía o transición espontánea y permite que el autómata **cambie de estado sin consumir ningún símbolo de la cadena de entrada**.

## Parte 2 — De NFA a DFA (Método de subconjuntos)

Instrucción:

Convierte el NFA obtenido en la parte anterior en su DFA equivalente,

siguiendo estos pasos:

1. Construye la tabla de subconjuntos paso a paso.

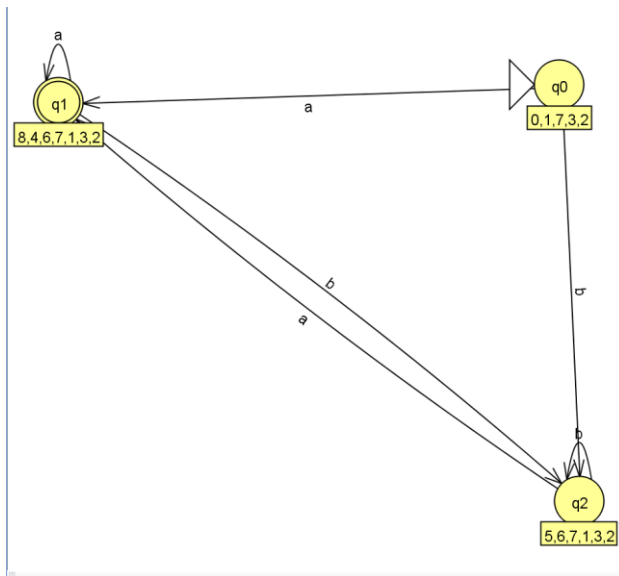
Tabla de subconjuntos:

| Estado DFA | a  | b  | $\lambda$  |
|------------|----|----|------------|
| q0         | -  | -  | q1,q7      |
| q1         | -  | -  | q2, q3, q6 |
| q2         | q4 | -  | -          |
| q3         | -  | q5 | -          |
| q4         | -  | -  | q6         |
| q5         | -  | -  | q6         |
| q6         | -  | -  | q1, q7     |
| q7         | q8 | -  | -          |
| q8         | -  | -  | -          |

2. Identifica los estados de aceptación.

| Estado DFA | Conjunto NFA                   | a  | b  | ¿Acepta? |
|------------|--------------------------------|----|----|----------|
| D0         | { q0, q1, q2, q3, q6, q7 }     | D1 | D2 | No       |
| D1         | { q1, q2, q3, q4, q6, q7, q8 } | D1 | D2 | Si       |
| D2         | { q1, q2, q3, q5, q6, q7 }     | D1 | D2 | No       |

### 3. Dibuja el diagrama del DFA final.



### Preguntas:

¿Cuántos estados tiene tu DFA en comparación con el NFA original?

El DFA tiene 3 estados a comparación del NFA que tiene 8 estados.

¿Por qué el DFA siempre tiene un número igual o mayor de estados?

El DFA siempre tiene un número igual o mayor de estados que el NFA porque cada estado del DFA representa un subconjunto de estados del NFA, asegurando que todas las posibles combinaciones de transiciones del NFA estén cubiertas de forma determinista. Mientras que un NFA puede moverse a varios estados con un mismo símbolo, un DFA necesita tener una transición única por símbolo desde cada estado, lo que a veces requiere crear nuevos estados que combinan varios estados del NFA. Por esta razón, el número de estados del DFA puede ser igual o incluso mayor, llegando en el peor caso a  $2^n$  estados si el NFA tiene  $n$  estados.

## Parte 3 — Minimización del DFA

### Instrucción:

Reduce el número de estados de tu DFA aplicando el método de particiones.

1. Clasifica los estados en grupos equivalentes.

Primero, dividimos los estados entre **aceptadores** y **no aceptadores**:

- Grupo 1 (Aceptación):  $\{ D_1 \}$
- Grupo 2 (No aceptación):  $\{ D_0, D_2 \}$

Analizamos si  $D_0$  y  $D_2$  son distinguibles:

| Estado | A                 | B          |
|--------|-------------------|------------|
| $D_0$  | $D_1$ (aceptador) | $D_2$ (no) |
| $D_2$  | $D_1$ (aceptador) | $D_2$ (no) |

Las transiciones de ambos estados apuntan a los mismos grupos ( $a \rightarrow$  aceptador,  $b \rightarrow$  no aceptador).

Por lo tanto,  **$D_0$  y  $D_2$  son equivalentes** y se pueden unir.

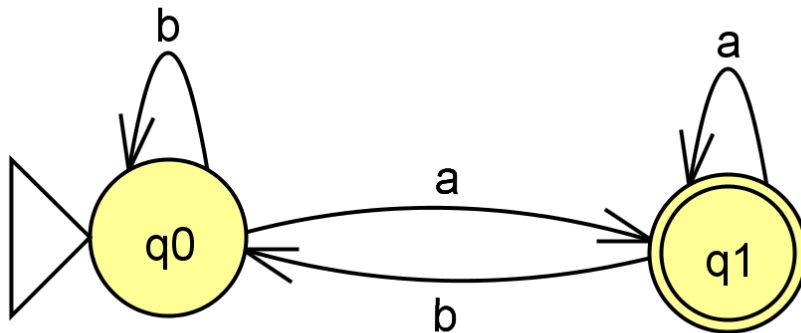
Nuevos grupos (DFA minimizado)

| Grupo | Estados del DFA original | ¿Acepta? |
|-------|--------------------------|----------|
| $M_0$ | $\{ D_0, D_2 \}$         | No       |
| $M_1$ | $\{ D_1 \}$              | Si       |

2. Construye la tabla de transición del DFA minimizado.

| Estado | A     | B     | ¿Acepta? |
|--------|-------|-------|----------|
| $M_0$  | $M_1$ | $M_0$ | No       |
| $M_1$  | $M_1$ | $M_0$ | Si       |

3. Dibuja el DFA mínimo.



Pregunta de análisis:

¿Por qué la minimización es útil en la etapa de análisis léxico de un compilador?

Porque permite tener menos estados, lo que lleva a tener menos estados redundantes y se ocupe un menor uso de memoria y rendimiento, haciéndolo más eficiente.

## Parte 4 — Implementación en código

Instrucción:

Programa el DFA minimizado obtenido en la Parte 3 para que acepte las cadenas del lenguaje:

$$L=(a|b)^*aL = (a|b)^*aL=(a|b)^*a$$

Usar C. El programa debe:

1. Leer una cadena del usuario.
2. Simular las transiciones del DFA.
3. Mostrar si la cadena es ACEPTADA o RECHAZADA.

Ejemplo de comportamiento esperado:

| Entrada | Resultado |
|---------|-----------|
|---------|-----------|

A  ACEPTADA

ba  ACEPTADA

bba  ACEPTADA

bb  RECHAZADA

Codigo:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {

    int transicion[2][2] = {
        {1, 0},
        {1, 0}
    };

    char cadena[100];
    int estado = 0;
    int entrada, i;

    printf("Ingrese la cadena: ");
    scanf("%s", cadena);

    for (i = 0; i < strlen(cadena); i++) {
        char c = cadena[i];

        if (c == 'a') {
            entrada = 0;
        } else if (c == 'b') {
            entrada = 1;
        } else {
            printf("Caracter no valido: %c\n", c);
            return 0;
        }

        estado = transicion[estado][entrada];
    }
}
```



```
if (estado == 1) {  
    printf("ACEPTADA\n");  
} else {  
    printf("RECHAZADA\n");  
}  
  
return 0;  
}
```

```
Ingrese la cadena: aaaa  
ACEPTADA
```

```
Ingrese la cadena: bbb  
RECHAZADA
```

```
Ingrese la cadena: bbbbbbba  
ACEPTADA
```

## Parte 5 — Reflexión final

Responde con tus palabras:

1. ¿Qué relación existe entre una expresión regular y el autómata que la representa?

Una expresión regular describe un patrón de cadenas, y el autómata es la forma de representarlo para que se puedan reconocer esas cadenas automáticamente. Básicamente, el autómata acepta exactamente las mismas cadenas que define la expresión regular.

2. ¿Por qué los compiladores prefieren trabajar con DFA en lugar de NFA?

Los compiladores prefieren DFA porque son deterministas: para cada símbolo solo hay un camino posible, lo que hace que sean más rápidos y eficientes que los NFA, que pueden tener varias opciones y generar ambigüedad.

3. ¿Cómo se integran los DFA en el analizador léxico de un compilador?

El analizador léxico convierte cada patrón de token en un DFA. Luego, al leer el código, el DFA sigue las transiciones y reconoce los tokens uno por uno de manera automática y rápida.